DeepPiC: xPU-PIM Cluster Architecture with Adaptive Resource-Aware Task Orchestration for DeepSeek-Style MoE Inference

Zixu Li[1], Manni Li[1], Zijian Huang[1], Jiayu Yang[1], Wending Zhao[1], Yinyin Lin[1]
Chengchen Wang[2], Haidong Tian[2] and Xiankui Xiong[2]
[1]*State Key Laboratory of Integrated Chips and Systems, School of Microelectronics, Fudan University*, Shanghai, China
[2]*State Key Laboratory of Mobile Multimedia Technology, ZTE Corporation*, Shenzhen, China
yylin@fudan.edu.cn

***Abstract*—The success of DeepSeek has driven demand for deploying high-performance inference clusters. However, due to its Transformer-based autoregressive structure, DeepSeek remains severely bandwidth-bound, limiting the scalability of traditional xPU (e.g., GPU/TPU). While DRAM-based processing-in-memory (PIM) offers a promising solution to overcome memory bottlenecks, its use in inference clusters for DeepSeek remains underexplored due to three challenges: (1) non-trivial inter-device communication overhead; (2) the need for expert parallelism in the mixture-of-experts (MoE) module; and (3) lack of efficient task offloading to PIM. To this end, we propose DeepPiC, a novel xPU-PIM cluster architecture designed for DeepSeek-style models with multi-latent attention (MLA) and MoE modules. DeepPiC introduces a heterogeneous xPU+HBM-PIM device to accelerate low arithmetic intensity operations. It can seamlessly replace conventional xPU devices without any modification to cluster-level interconnect topology. However, DeepPiC cannot fully realize its performance potential under static scheduling, which fails to adapt to shifting compute and memory demands driven by multidimensional variability (model heterogeneity, cluster-scale volatility, runtime dynamics). This induces inter-device communication overhead and intra-device underutilization. Thus, we propose Adaptive Resource-Aware Task Orchestration (ARTO), a two-phase strategy that decouples global model partitioning from local task assignment by dynamically coordinating (1) cross-device parallelism optimization and (2) intra-device xPU/PIM mapping. Evaluated on DeepSeek V3-671B using H20-, A100-, and H200-Cluster (H20 serves as a compute-limited alternative to high-end GPUs), DeepPiC (H20+HBM-PIM) achieves up to 3×, 2× and 1.3× speedup over H20-, A100-, and H200-Cluster at small batch sizes, while maintaining 74% and 54% of A100- and H200-Cluster performance at large batch sizes. These results demonstrate that DeepPiC enables low-end xPU to approach or even exceed premium ones by fundamentally overcoming memory bottlenecks via adaptive scheduling that orchestrates PIM and xPU heterogeneous resources.**

***Index Terms*—Processing-in-Memory (PIM), DRAM, Large Language Model (LLM), Mixture-of-Experts (MoE), Cluster**

I. Introduction

The success of the DeepSeek model [1] has sparked widespread interest in deploying large language models (LLMs) on inference clusters. Increasingly, organizations and individual users seek to run these models on proprietary datasets using their own on-premise inference clusters to support application-specific requirements. Although the development of the DeepSeek model is guided by hardware-aware design [2], it fundamentally inherits the Transformer archite-cture [3], whose autoregressive computation pattern remains severely bandwidth-bound—e.g., GPT-3 175B spends up to 92% of its time on memory-intensive operations [4]. Consequently, simply scaling compute resources alone fails to deliver proportional performance improvements.

Recently, DRAM-based processing-in-memory (PIM) has emerged as a promising architectural solution to address memory bandwidth bottlenecks in modern AI workloads. By integrating compute units near memory arrays, PIM significantly enhances effective memory bandwidth while reducing access latency and energy consumption [5]. For instance, FIM [6], [7] embeds PIM execution units (PCU) within the high bandwidth memory (HBM) DRAM die, achieving up to 11.2× performance improvement for general matrix-vector multiplication (GEMV) operations. Similarly, AiM [8]–[10] integrates PCUs near GDDR6 banks, resulting in a 9.2× speedup. In the past few years, Samsung and SK hynix—two of the world's largest DRAM manufacturers—have made tangible progress in deploying DRAM-based PIM technologies to LLM inference, particularly for accelerating memory-bound operations such as GEMV [4], [9]–[11].

However, the potential benefits of deploying PIM in inference clusters remain largely underexplored, primarily due to the following three key challenges: (1) Cross-device communication overhead: In large-scale inference clusters, inter-device communication latency is non-negligible. While PIM can effectively reduce memory access latency within a single device, it does not address communication overhead across devices. As a result, whether PIM can deliver meaningful cluster-level performance gains remains unclear and requires systematic quantitative evaluation. (2) Parallelism strategy for mixture-of-experts (MoE): The DeepSeek model incorporates an MoE module, which demands expert parallelism (EP) — a cluster scheduling strategy that deviates from traditional tensor parallelism (TP), data parallelism (DP), or pipeline parallelism

This work was supported by ZTE industry-university-institute cooperation funds under grant NO.IA20241120003-PO002.

(PP). Although prior works such as Calculon [12] explored TP, DP, and PP strategies for LLM training, and DynamoLLM [13], SmartMoE [14], and FastMoE [15] investigated EP for MoE training, these studies primarily focus on training scenarios and do not consider the integration of PIM in cluster deployments. (3) Lack of efficient task offloading methods to determine which computations should be offloaded to PIM: When xPU and PIM are integrated within a single device, scheduling becomes increasingly complex—raising the question of how to effectively map different tasks to the appropriate computing unit based on their characteristics. While existing works like IANUS [16], Attacc [17], NeuPIM [18], and GPOS [19] have proposed intra-device scheduling mechanisms, these efforts typically target standalone devices and neglect cluster-level deployment, especially under the constraints of MoE's expert parallelism during inference.

To address the above challenges, this paper makes the following key contributions:

- We conduct the first in-depth analysis of the potential for PIM to accelerate LLMs, with a focus on the DeepSeek model, in inference clusters.
- We propose DeepPiC, a novel xPU-PIM cluster architecture designed for DeepSeek-style models. At the device level, DeepPiC introduces a heterogeneous xPU+HBM-PIM device to improve the execution efficiency of low arithmetic intensity operations. This design can seamlessly replace conventional xPU devices in existing clusters without any modification to the cluster-level interconnect topology.
- Static scheduling limits DeepPiC's effectiveness by failing to adapt to multidimensional variability (model heterogeneity, cluster-scale volatility, runtime dynamics), leading to communication overhead across devices and poor utilization within device. To address this, we propose Adaptive Resource-Aware Task Orchestration (ARTO), a two-phase strategy that decouples global placement from local scheduling by dynamically coordinating parallelism across devices and task mapping to xPU or PIM within each device.

We evaluate DeepPiC (H20+HBM-PIM) on DeepSeek V3-671B using H20-, A100-, and H200-Clusters, where H20 serves as a throughput-limited, cost-efficient alternative to high-end GPUs. DeepPiC (H20+HBM-PIM) achieves up to 3×, 2× and 1.3× speedup over H20-, A100-, and H200-Cluster at small batch sizes, while maintaining 74% and 54% of A100- and H200-Cluster performance at large batch sizes. DeepPiC effectively boosts the performance of low-end xPUs, enabling them to rival or exceed that of top-tier xPUs by utilizing PIM.

II. Motivation: PIM Opportunity for the DeepSeek Model in Inference Clusters

The evolution of the DeepSeek model has been guided by hardware-aware design principles [1], [2], as shown in Fig. 1. However, during inference deployment on clusters,

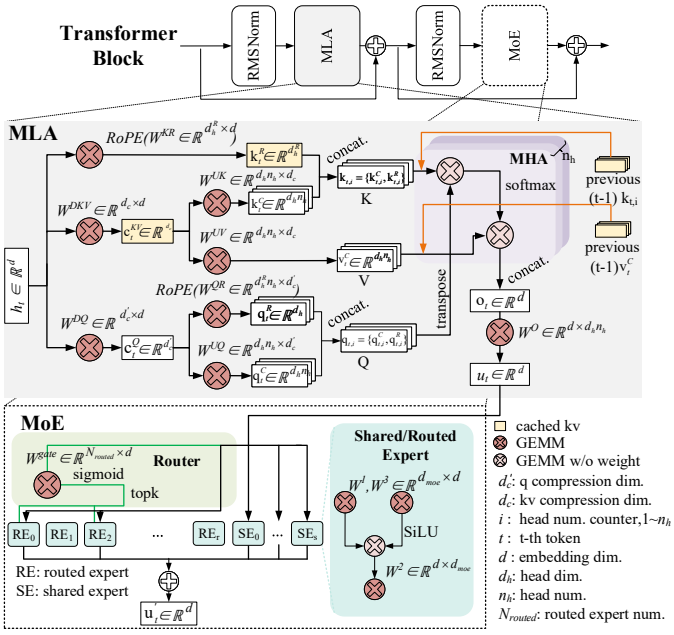

Fig. 1. Hardware-aware architecture of the DeepSeek V3 model, featuring MLA for memory-efficient latent projection and MoE for scaling model capacity while maintaining computational efficiency. In the decoding stage, all GEMM are memory-bound at small batch sizes, while GEMM without weight remain memory-bound regardless of batch size.

the decode phase of the model remains fundamentally bottlenecked by hardware bandwidth. Specifically, limited interconnect bandwidth hampers efficient communication between devices, while restricted memory bandwidth impedes data access during computation.

Fig. 2(a) illustrates the typical parallelism strategies employed in DeepSeek inference cluster deployment, including DP, TP and EP. Among these, TP and EP incur non-negligible cross-device communication overhead. To improve inference performance, DeepSeek adopts a DeepEP [20] strategy that overlaps computation with EP communication, as shown in Fig. 2(b). Under this overlapping scheme, the latency of general matrix-matrix multiplication (GEMM) becomes one of the dominant contributors to the total execution time. This is further confirmed by the latency breakdown in Fig. 3, which evaluates various parallel strategy combinations (with detailed configurations described in Section 5.1). In all evaluated scenarios, EP communication is fully hidden behind computation, while TP communication remains only partially overlapped. Notably, when the parallel strategy is well tuned, the TP communication overhead becomes relatively small—highlighting that the majority of execution time is now dominated by GEMM operations.

The latency of GEMM, which dominates computation time in the decode phase, is particularly susceptible to memory bandwidth constraints. For example, GEMM in the MLA module has bandwidth bottlenecks regardless of batch size due to the autoregressive computation pattern; and GEMM in the MoE module is bandwidth-bound when the batch size is small. This is because, although modern xPUs can

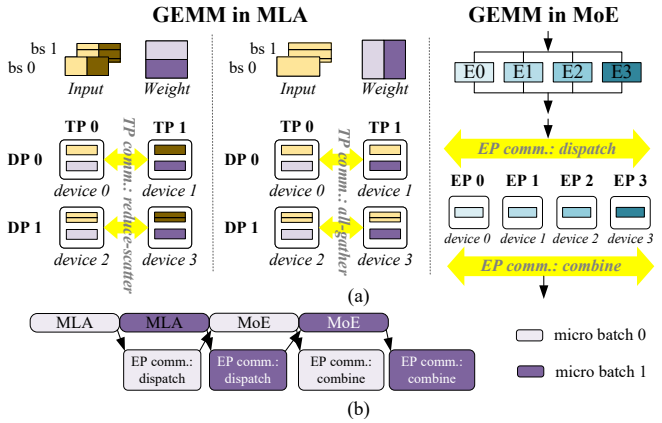

Fig. 2. (a) Typical parallel strategies for the MLA and MoE modules in DeepSeek v3 inference cluster deployment. TP and EP introduce inter-device communication overhead. (b) The DeepEP strategy employed by DeepSeek overlaps computation with EP communication to improve inference performance.

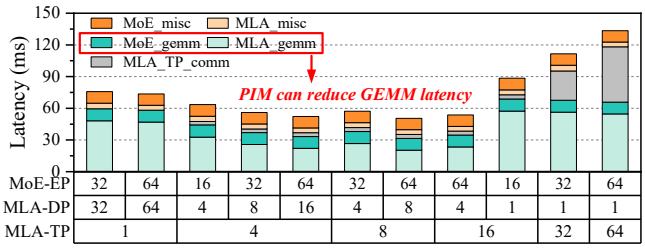

Fig. 3. Latency breakdown under various parallel strategy combinations for DeepSeek V3 at batch size = 64. GEMM emerges as the dominant bottleneck under the optimal parallel strategy (MLA-TP=8, MLA-DP=8, MoE-EP=64).

deliver several hundred TFLOPS to keep up with the rapidly growing LLMs, advances in HBM bandwidth and capacity have lagged behind. Fig. 4(a) and 4(b) illustrates a GEMM time breakdown under DRAM bandwidth constraints. It clearly shows significant computation bubbles, where compute units remain idle while waiting for data. In such cases, memory access latency dominates the overall GEMM execution time.

PIM integrates computation closer to memory, offering high bandwidth and ultra-low access latency, albeit with lower compute throughput compared to xPUs [19]. This architecture effectively alleviates the memory bottlenecks that slow down GEMM, resulting in a marked reduction in overall inference time, as shown in Fig. 4(c) and 4(d). As such, PIM presents a promising architectural solution for accelerating inference cluster in large-scale DeepSeek-style models.

III. DeepPiC Architecture

Fig. 5 illustrates the DeepPiC architecture, which adopts a two-level hierarchical design (device-level and cluster-level) to optimize LLM inference under bandwidth constraints.

A. Device-level: xPU+HBM-PIM

To coordinate between compute-bound and memory-bound tasks within a single device, DeepPiC re-architects the conventional xPU device into a heterogeneous xPU+HBM-PIM device. HBM is selected as the substrate for PIM integration due to its significantly higher bandwidth and widespread adoption in high-performance clusters, making it a more suitable choice than DDR or GDDR. The proposed HBM-PIM maintains full compatibility with standard HBM interfaces, ensuring seamless integration into existing infrastructures.

The HBM-PIM module adopts a stacked structure consisting of a buffer die and multiple DRAM dies connected via TSVs. Half of the DRAM dies integrate PIM compute units (PCUs), while the rest retain standard DRAM arrays. Each PCU is shared by two banks, with each bank occupying half its original area to reduce silicon footprint and power.

While this integration feasibility has been demonstrated in prior commercial PIM systems [6], [7], [11], DeepPiC introduces a redesigned PCU datapath optimized for LLM inference. In contrast to prior PIM designs that typically support 16-way 16-bit SIMD execution, DeepPiC enhances the PCU to support 32-way 8-bit SIMD within the same 256-bit datapath. This improvement matches the hybrid precision (BF16+FP8) widely adopted in DeepSeek-style models, delivering better model–hardware compatibility and improved throughput. Each PCU integrates a single-instruction multiple-data (SIMD) floating-point unit (FPU), control logic, and three types of register files: command (CRF), general-purpose (GRF), and scalar (SRF).

B. Cluster-level of DeepPiC

At the cluster level, DeepPiC replaces traditional xPU-based devices with heterogeneous xPU+HBM-PIM devices introduced in Section 3.1, without requiring modifications to system organization or interconnect topology while enabling new forms of memory-centric acceleration. This design is compatible with a wide range of existing and future infrastructures, such as NVIDIA's DGX/HGX series, Huawei's CloudMatrix 384. DeepPiC supports common node-scale configurations, such as 8-device nodes (e.g., DGX-A100 [21]), 16-device nodes (e.g., DGX-2 [22]), and large-scale 72-device configurations (e.g., B200 with third-generation NVLink [23]). Furthermore, DeepPiC is compatible with Huawei's latest CloudMatrix 384 [24], which connects 384 devices across 48 nodes using an optical unified-bus interconnect.

Fig. 5 illustrates an example of the DeepPiC cluster architecture based on an 8-device node. Nodes are interconnected via InfiniBand (IB), each equipped with dedicated network interface controllers (NICs) to support cross-node communication. Within each node, devices are connected using high-bandwidth all-to-all interconnects such as NVLink, which offer significantly higher bandwidth than IB.

IV. Adaptive Resource-Aware Task Orchestration (ARTO)

Static scheduling fails to unleash the potential of DeepPiC architecture, as it cannot adapt to multi-dimensional variability inherent in LLM inference cluster. This variability stems from model heterogeneity (MLA vs. MoE), cluster-scale volatility

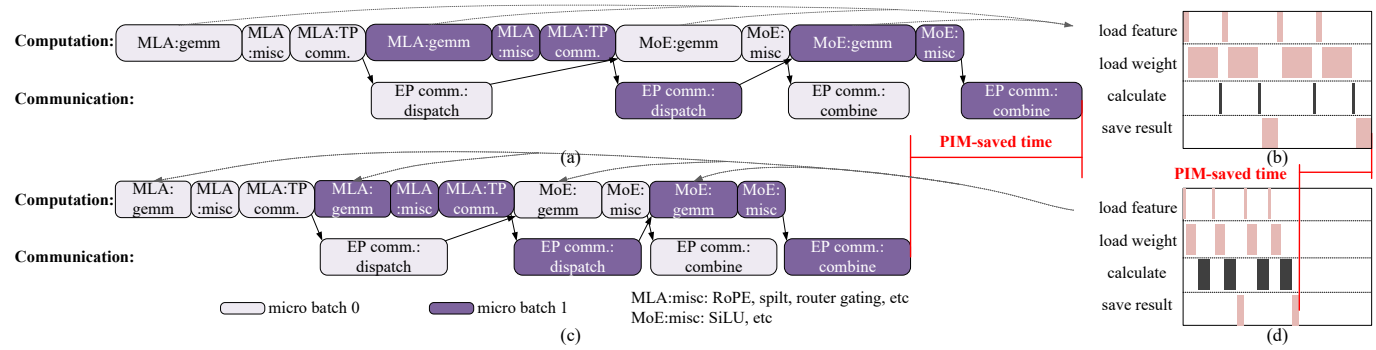

Fig. 4. (a) Execution time breakdown under the DeepSeek's overlapping strategy without PIM and its (b) GEMM latency breakdown showing DRAM accessing latency (pink) dominates due to bandwidth bottlenecks. (c) Execution time breakdown under the overlapping strategy with PIM and its (d) GEMM latency breakdown with PIM, where DRAM access time (pink) is significantly reduced, improving inference speed.

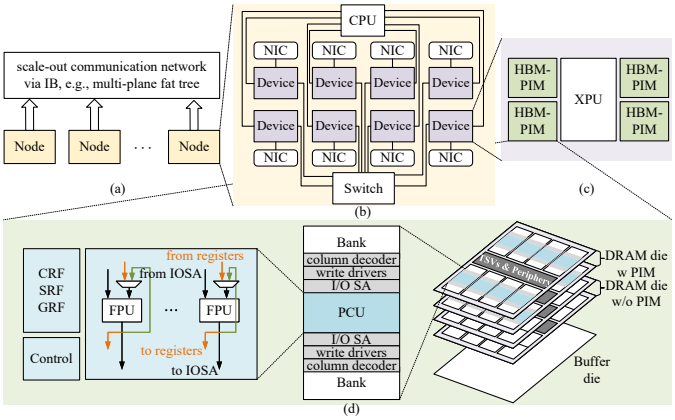

Fig. 5. DeepPiC architecture overview. (a) Cluster-level system design. (b) Example of an 8-device node. (c) Device-level architecture: heterogeneous xPU+HBM-PIM device. (d) Internal structure of HBM-PIM module.

(device count), and runtime dynamics (batch size and prefill vs. decode stages)—all of which impose varying demands on compute and memory resources. Such mismatch hampers efficient task orchestration, causing increased inter-device communication overhead (Fig. 3) and underutilization of intra-device resources (<40% [18]).

Thus, we propose ARTO, a strategy that dynamically partitions and schedules tasks based on current resource demands. It operates in two phases, jointly determining how to divide the model across devices and how to assign tasks within each device to xPU or PIM, aiming to minimize end-to-end inference latency. A key challenge lies in the tight coupling between these two phases: optimizing device placement alone overlooks intra-device efficiency, while task offloading decisions within device depend on global partitioning. ARTO decouples these concerns into two mutually informed steps, enabling adaptive yet efficient orchestration.

A. Phase 1: Parallelism Strategy across Devices

Phase 1 adaptively determines parallelism strategy combination (TP, DP, EP) based on the model's module type (MLA vs. MoE) and the current cluster scale (available devices), as detailed in Algorithm 1. The determined parallel configuration defines how the model is partitioned across devices at the global level.

Rather than applying a static parallelism scheme, Algorithm 1 explores valid combinations of parallel configurations under per-device memory and compute constraints, and selects the one that minimizes end-to-end latency, particularly reducing inter-device communication overhead.

B. Phase 2: Task Assignment within Device

Phase 2 focuses on fine-grained scheduling of tasks within each heterogeneous device to maximize utilization of both xPU and HBM-PIM resources. During inference, resource demands vary significantly with the stage (prefill vs. decode) and batch size. A static, fixed task assignment leads to inefficient use of device resources. For example, the MoE workload shifts from being compute-bound to memory-bound as batch size increases, prompting a change in execution location from xPU to PIM.

To address this, we implement dynamic task offloading based on varying workload characteristics, as illustrated in Fig. 6. Specifically, we analyze the current compute and memory resource requirements by progressively examining the execution stage and batch size.

Furthermore, Phase 2 supports four flexible execution modes that allow xPU and PIM either serially or concurrently, depending on task dependencies and resource demands. As shown in Fig. 7, this design allows xPU and HBM-PIM to handle different workload types in parallel, potentially enhancing overall inference efficiency.

V. Experimental Results

A. Methodology

The proposed DeepPiC (H20+HBM-PIM) architecture is compared against three GPU-only cluster baselines—H20-Cluster, A100-Cluster [21], and H200-Cluster [25]—listed in ascending order of compute capability. H20 serves as a compute-limited, cost-efficient alternative to high-end GPUs. Detailed hardware specifications are summarized in Table I. We develop a performance simulator for DeepPiC to evaluate

Algorithm 1: Parallelism Strategy Search

Input : M: DeepSeek-style model (MLA + MoE);
L: max sequence length;
B: batch size;
N_{dev}: number of available devices;
C_{mem}: per-device memory capacity
Output : Optimal parallel strategy $(\text{TP}^*, \text{DP}^*, \text{EP}^*)$
Params: S_{moe}: MoE weights size;
S_{kv}: attention KV-cache size

$\text{ep}_{\min} \leftarrow \lceil S_{\text{moe}}/C_{\text{mem}} \rceil$;
$B_{\max} \leftarrow \lfloor C_{\text{mem}}/S_{\text{kv}} \rfloor$;
Generate candidate (TP, DP) pairs s.t. $\text{TP} \cdot \text{DP} \leq N_{\text{dev}}$;
Valid_configurations $\leftarrow \emptyset$;
foreach (TP, DP) *in candidate MLA configs* **do**
  if $B \bmod (TP \cdot DP) \neq 0$ **then continue**;
  if $B/DP > B_{\max}$ **then continue**;
  MLA_device_num $\leftarrow \text{TP} \cdot \text{DP}$;
  EP $\leftarrow \text{TP} \cdot \text{DP}$;
  if $B \bmod EP \neq 0$ ***or*** $EP < ep_{\min}$ **then continue**;
  Add $(\text{TP}, \text{DP}, \text{EP})$ to Valid_configurations;
end
Best_latency $\leftarrow \infty$;
foreach (TP, DP, EP) *in Valid_configurations* **do**
  latency $\leftarrow$ Simulate$(M, \text{TP}, \text{DP}, \text{EP}, L, B)$;
  if *latency* $<$ *Best_latency* **then**
    Best_latency $\leftarrow$ latency;
    $(\text{TP}^*, \text{DP}^*, \text{EP}^*) \leftarrow (\text{TP}, \text{DP}, \text{EP})$;
  end
end
return $(\text{TP}^*, \text{DP}^*, \text{EP}^*)$;

TABLE I
GPU-CLUSTER AND DEEPPIC HARDWARE SPECIFICATIONS.

Parameters per device	H20-Cluster	A100-Cluster	H200-Cluster	DeepPiC (H20+HBM-PIM)
DRAM I/O bandwidth (TB/s)	3.9	2.039	4.8	3.9
DRAM compute bandwidth (TB/s)	-	-	-	19.67
HBM stack number	6	5	6	6
DRAM capacity (GB)	96	80	141	72
Throughput-FP8 (TFLOPS)	296	624	3341	296+19.7
HBM-PIM Configurations in DeepPiC				
pCHs/stack, Banks/pCH, PCU/pCH				16,16,8
tRP=14, tRCD=14, tRAS=34, tRRD_L=6, tWR=16, tCCD_S=1, tCCD_L=2, tREFI=3900, tRFC=260, tFAW=30				

its inference performance. This simulator extends the open source Calculon cluster simulator [26] with new abstractions for DeepSeek-style models, including support for expert parallelism and its associated all-to-all communication overhead. To accurately model the heterogeneous xPU+HBM-PIM device in DeepPiC, we modify Ramulator [27], incorporating HBM-PIM modeling inspired by the open-source simulator Attacc [28]. We adopt DeepSeek V3-671B as our target model. The

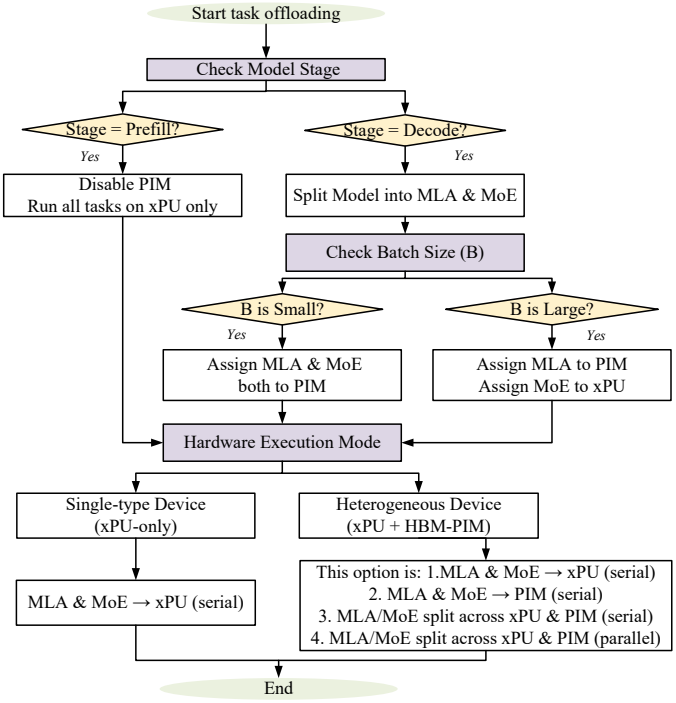

Fig. 6. The workflow of task assignment within device.

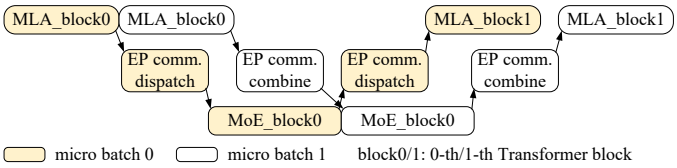

Fig. 7. Parallel execution of MLA and MoE on DeepPiC's heterogeneous xPU + HBM-PIM device.

input and output sequence lengths are set to 500 and 200, respectively, with a maximum sequence length of 4096.

B. Performance Evaluation

Fig. 8 presents the latency breakdown of DeepPiC (H20+ HBM-PIM), H20-, A100-, and H200-Cluster under various valid cluster-level parallel strategies. Two key observations can be drawn: (1) DeepPiC significantly outperforms GPU-only clusters, achieving up to 3×, 2× and 1.3× speedup over H20-, A100-, and H200-Cluster on average. This is because that both MLA and MoE modules are bandwidth-bound at small batch sizes, where compute resources are underutilized. In such cases, the HBM-PIM component in DeepPiC effectively reduces memory access latency, while stronger compute power alone (as in A100, H200) provides limited benefit. Consequently, DeepPiC can even outperform the high-end A100- and H200-Cluster at memory-bound scenario. (2) Comparative analysis of end-to-end latency under diverse parallelism strategies clearly demonstrates ARTO's superiority. Notably, even on the same high-performance DeepPiC architecture, suboptimal static configurations can severely degrade performance. For example, at batch size of 64 with the same number of

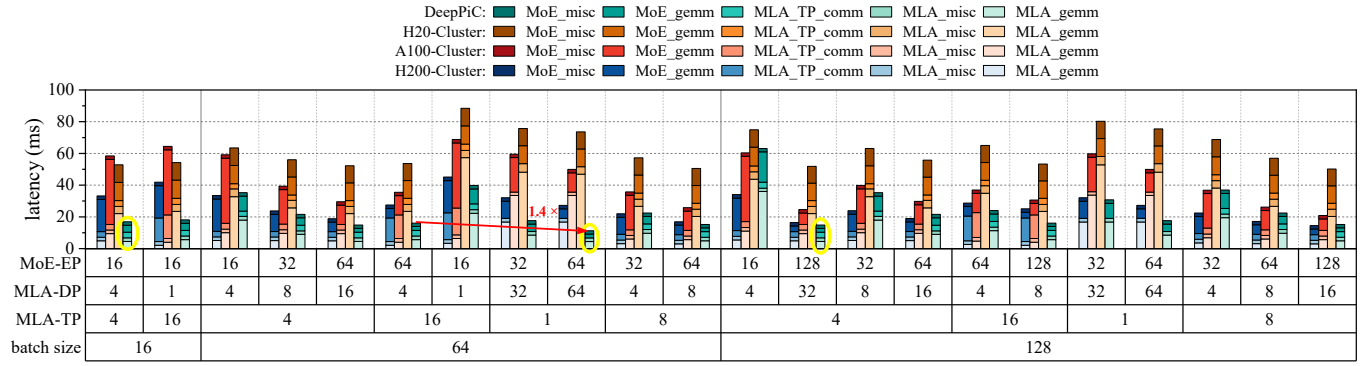

Fig. 8. Latency breakdown comparison across DeepPiC (H20+HBM-PIM), H20-, A100-, and H200-Cluster under varying cluster-level parallelism configurations. Yellow circles indicate the optimal scheduling points selected by ARTO for each batch size on DeepPiC.

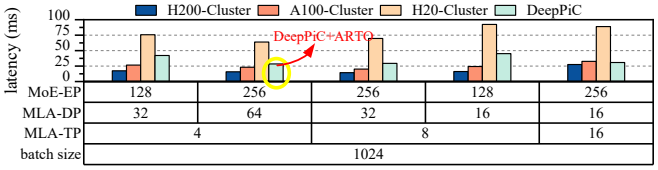

Fig. 9. Comparison of latency across DeepPiC (H20+HBM-PIM), H20, A100, and H200-Cluster at batch size = 1024.

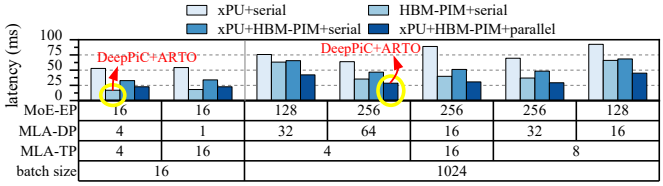

Fig. 10. Latency of ARTO's dynamically selected execution modes across batch sizes (16 vs. 1024).

devices, a poorly tuned static setup (MLA-TP, MLA-DP, MoE-EP = 16, 4, 64) results in 1.4× higher latency compared to the ARTO-optimized configuration (MLA-TP, MLA-DP, MoE-EP = 1, 64, 64). This stark contrast proves that hardware advantages are nullified without adaptive task orchestration.

Fig. 9 shows the latency comparison among DeepPiC, H20-, A100-, and H200-Cluster when the batch size increases to 1024. In this setting, DeepPiC achieves a 2.3× speedup over H20-Cluster and reaches 74% and 54% of A100- and H200-Cluster performance, respectively. In contrast, H20-Cluster achieves only 33% and 23% of the performance of the A100- and H200-Clusters.

As batch size scales, the MoE module undergoes a critical transition in resource bottlenecks—from memory bandwidth saturation to compute intensiveness—while MLA persists as a bandwidth-bound workload. ARTO autonomously adapts to this dynamic shift through two-phase optimization: (1) an optimal parallelism configuration across devices, and (2) the parallel execution mode (*xPU+HBM-PIM+parallel*) within devices. This enables MoE to fully exploit xPU's compute capacity and MLA to maintain peak efficiency on PIM. Consequently, ARTO enables DeepPiC to sustain over 70% and 50% of A100 and H200 performance even under large batch sizes.

We further dissect how ARTO Phase 2 intelligently navigates hardware execution modes under batch-size-driven resource dynamics. Fig. 10 reveals latency disparities across four execution modes, where ARTO demonstrates adaptation: at small batch size (e.g., 16), ARTO activates *HBM-PIM+serial* mode, exploiting PIM's memory efficiency to minimize latency. At large batch size (e.g., 1024), ARTO switches to *xPU+HBM-PIM+parallel* mode, concurrently processing compute-intensive MoE on xPU and memory-bound MLA on PIM. This mode further benefits from execution overlap, reducing end-to-end latency. These results showcase the core advantage of DeepPiC+ARTO: fundamentally overcoming memory bottlenecks through adaptive scheduling that orchestrates heterogeneous resources (HBM-PIM and xPU) for workload-optimized acceleration—transforming traditional bandwidth constraints into scalable performance opportunities.

VI. Conclusion

This paper proposes DeepPiC, a novel xPU–PIM cluster architecture for DeepSeek-style LLM that overcomes three key barriers: (1) inter-device communication overhead, (2) MoE expert parallelism, and (3) PIM task offloading inefficiency. By integrating heterogeneous xPU+HBM-PIM devices, DeepPiC alleviates memory bottlenecks while maintaining cluster compatibility. To unleash its potential, we introduce ARTO, dynamically decoupling global model partitioning (Phase 1) and local xPU/PIM mapping (Phase 2). Evaluations on DeepSeek V3-671B show DeepPiC (H20+HBM-PIM) surpasses H20/A100/H200-Clusters by 3×/2×/1.3× at small batches, and maintains 74%/54% of A100/H200 performance at scale—enabling low-end xPUs to rival premium hardware by fundamentally overcoming memory bottlenecks via adaptive scheduling that orchestrates PIM and xPU heterogeneous resources.

References

[1] DeepSeek-AI, A. Liu, B. Feng, B. Xue *et al.*, "DeepSeek-V3 Technical Report," Feb. 2025.

[2] C. Zhao, C. Deng, C. Ruan *et al.*, "Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures," May 2025.

[3] A. Vaswani, N. Shazeer, N. Parmar *et al.*, "Attention Is All You Need," Aug. 2023.

[4] Y. Kwon, "Cost effective LLM inference solution using SK hynix's AiM (Accelerator-in-Memory)," in *SC23*, 2023.

[5] M. He, C. Song, I. Kim *et al.*, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *Proc. IEEE/ACM Int. Symp. Microarchit. (MICRO)*. IEEE, Oct. 2020, pp. 372–385.

[6] Y.-C. Kwon, S. H. Lee, J. Lee *et al.*, "25.4 A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications," *ISSCC*, 2021.

[7] S. Lee, M. B. Division, S. Electronics *et al.*, "Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product," in *IEEE/ACM Symp. Comput. Archit. (ISCA)*, 2021, pp. 43–56.

[8] D. Kwon, S. Lee, K. Kim *et al.*, "A 1ynm 1.25V 8Gb 16Gb/s/Pin GDDR6-Based Accelerator-in-Memory Supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep Learning Application," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 291–302, Jan. 2023.

[9] Y. Kwon, K. Vladimir, N. Kim *et al.*, "System Architecture and Software Stack for GDDR6-Aim," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. Cupertino, CA, USA: IEEE, Aug. 2022, pp. 1–25.

[10] Y. Kwon, G. Kim, N. Kim *et al.*, "Memory-Centric Computing with SK hynix's Domain-Specific Memory," in *Hot Chips*, 2023.

[11] J. H. Kim, Y. Ro, J. So *et al.*, "Samsung PIM/PNM for Transformer based AI: Energy Efficiency on PIM/PNM Cluster," in *Hot Chips*, 2023.

[12] M. Isaev, N. Mcdonald, L. Dennison *et al.*, "Calculon: A methodology and tool for high-level co-design of systems and large language models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Denver CO USA: ACM, Nov. 2023, pp. 1–14.

[13] J. Stojkovic, C. Zhang, Í. Goiri *et al.*, "DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Mar. 2025, pp. 1348–1362.

[14] M. Zhai, J. He, Z. Ma *et al.*, "SmartMoE: Efficiently training Sparsely-Activated models through combining offline and online parallelization," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 961–975.

[15] J. He, J. Qiu, A. Zeng *et al.*, "FastMoE: A Fast Mixture-of-Expert Training System," Mar. 2021.

[16] M. Seo, X. T. Nguyen, S. J. Hwang *et al.*, "IANUS: Integrated Accelerator based on NPU-PIM Unified Memory System," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24, vol. 3. New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 545–560.

[17] J. Park, J. Choi, K. Kyung *et al.*, "AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24, vol. 2. New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 103–119.

[18] G. Heo, S. Lee, J. Cho *et al.*, "NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24, vol. 3. New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 722–737.

[19] Z. Li, W. Wang, M. Li *et al.*, "GPOS: A General and Precise Offloading Strategy for High Generality of DNN Acceleration by OCP and NDP Co-optimizing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2025.

[20] Deepseek-Ai, "GitHub - deepseek-ai/DeepEP: DeepEP: An efficient expert-parallel communication library," May 2025. [Online]. Available: https://github.com/deepseek-ai/DeepEP

[21] NVIDIA, "NVIDIA DGX A100 | The Universal System for AI Infrastructure," 2020. [Online]. Available: https://images.nvidia.com/aem-dam/Solutions/Data-Center/nvidia-dgx-a100-datasheet.pdf

[22] NVIDIA., "Dgx-2/2h system," 2018. [Online]. Available: https://docs.nvidia.com/dgx/pdf/dgx2-user-guide.pdf

[23] NVIDIA, "NVIDIA DGX B200 Datasheet," 2024. [Online]. Available: https://resources.nvidia.com/en-us-dgx-systems/dgx-b200-datasheet

[24] P. Zuo, H. Lin, J. Deng *et al.*, "Serving Large Language Models on Huawei CloudMatrix384," Jun. 2025.

[25] NVIDIA, "NVIDIA H200 Tensor Core GPU Supercharging AI and HPC workloads." 2024. [Online]. Available: https://applieddatasystems.com/wp-content/uploads/2024/08/hpc-datasheet-sc23-h200-datasheet-3002446.pdf

[26] Georgia Institute of Technology and NVIDIA, "Calculon-Ai/Calculon," May 2025. [Online]. Available: https://github.com/calculon-ai/calculon

[27] S. R. G. at ETH Zurich and C. M. University, "CMU-SAFARI/ramulator2," Jun. 2025. [Online]. Available: https://github.com/CMU-SAFARI/ramulator2

[28] S. N. University, "Scale-snu/attacc_simulator," Jun. 2025. [Online]. Available: https://github.com/scale-snu/attacc$_simulator$